



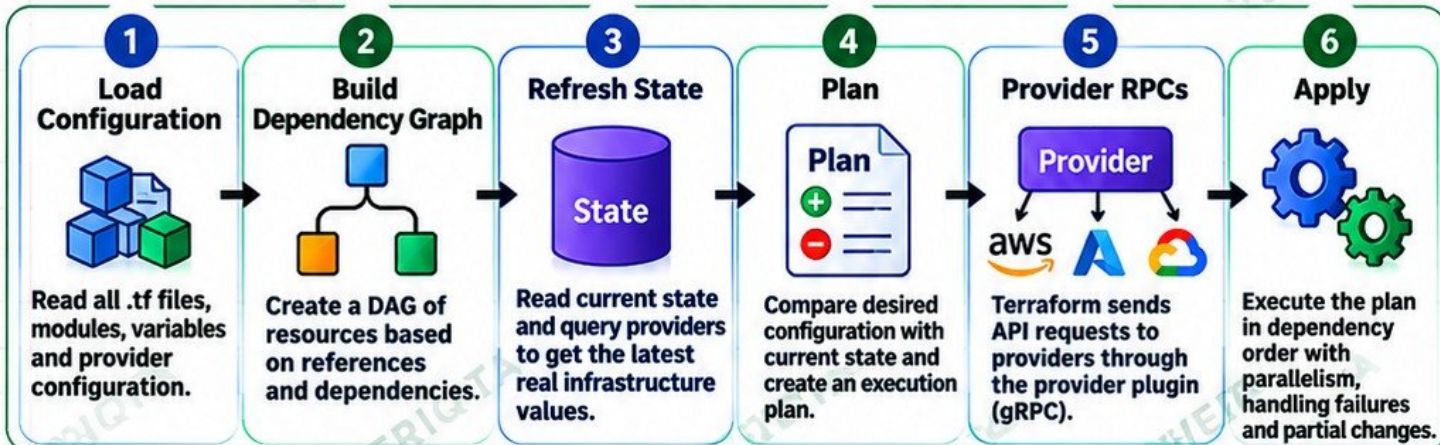
Terraform for Senior Infrastructure Engineers



01



Terraform Execution Model Under the Hood



1 Configuration Loading

- Terraform reads all .tf files in the working directory and modules.
- Resolves variables, locals, data sources and provider configuration.
- Builds an internal representation of the desired infrastructure.

```

terraform init    # install providers
terraform validate # check configuration
  
```

2 Dependency Graph

- Terraform builds a directed acyclic graph (DAG) of all resources, data sources and modules.
- Implicit dependencies from references (e.g. resource.id).
- Explicit dependencies using `depends_on`.
- Independent resources run in parallel.

```

resource "aws_instance" "app" {
  subnet_id = aws_subnet.private.id # implicit dependency
}
resource "aws_eip" "ip" {
  depends_on = [aws_instance.app] # explicit dependency
}
  
```

3 Refresh and State Reads

- Terraform reads the current state file.
- Queries providers to get the latest real infrastructure values (refresh).
- Detects drift between state and real resources.

```

terraform plan # runs refresh automatically
terraform plan -refresh-only # check drift only
  
```

4 Planning

- Compares desired configuration with refreshed state.
- Identifies create, update, destroy actions.
- Marks attributes as known, unknown or computed.
- Shows a change plan without making changes.

```

terraform aplan
terraform show -json plan.out # machine readable plan
  
```

5 Provider RPCs

- Terraform uses provider plugins to communicate with cloud APIs (gRPC).
- Each provider handles authentication, API calls, retries and error handling.
- Provider behavior depends on API limits, eventual consistency and service-specific rules.

6 Apply Execution

- Executes the plan in dependency order.
- Runs independent resources in parallel (default: 10).
- Stops on errors by default.
- Can result in partial changes if a failure occurs.

```

terraform apply # apply the plan
terraform apply -parallelism=20 # increase parallelism
  
```

? Unknown Values

- Some values are unknown during planning (e.g. IDs).
- Marked as (known after apply) in the plan.
- Depend on provider API responses at apply time.
- Normal and expected, depecially for computed attributes.

! Partial Failure

- If an error occurs, some resources may be created/updated and others may fail.
- State will reflect the successful changes.
- You must investigate and fix, then re-run apply.
- Design for idempotency and safe recovery.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

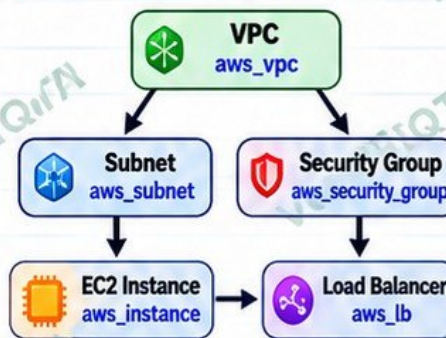
Dependency Graphs and Resource Ordering



1 How Terraform Builds the Graph

- Terraform parses configuration and builds a **Directed Acyclic Graph (DAG)** of all resources.
- Implicit dependencies** are created automatically when a resource references another resource.
- Explicit dependencies** can be added using `depends_on` when needed.
- Terraform uses the graph to determine the correct creation, update and destroy order.

2 Example Dependency Graph



- VPC is created first
- Subnet and Security Group in parallel (depend on VPC)
- EC2 depends on Subnet and SG
- Load Balancer depends on EC2

3 Implicit vs Explicit Dependencies

Implicit Dependency (recommended)

```
resource "aws_instance" "web" {
  subnet_id = aws_subnet.app.id
  security_groups = [
    aws_security_group.web.id
  ]
}
```

- Automatically creates the dependency in the graph.
- Preferred and usually sufficient.

Explicit Dependency (use only when needed)

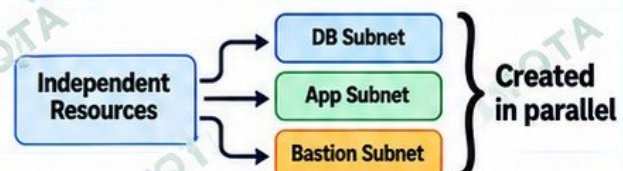
```
resource "aws_instance" "web" {
  ami = "ami-12345"
  instance_type = "t3.micro"
  depends_on = [
    aws_security_group.web
  ]
}
```

- For cases with no direct reference but a real dependency exists.
- Overuse can make the graph slower and harder to maintain.

4 Parallelism in the Graph

- Terraform executes independent resources in parallel (default parallelism: 10).
- Resources with no dependency can be created at the same time.
- You can adjust parallelism if needed.

```
terraform apply -parallelism=20
```



5 Hidden Dependencies

- Some dependencies are not obvious.
- Examples: IAM roles, permissions, DNS records, service quotas, or resources created outside Terraform.
- These can cause failures even when the graph looks correct.



Example

- EC2 may require an IAM role with specific permissions.
- RDS may require a subnet group and parameter group.
- An application may depend on a DNS record to be created.

6 Cycles in the Graph

- A cycle happens when resources depend on each other directly or indirectly.
- Terraform cannot create a cyclic graph.
- You will see an error: **Cycle: ...**



Cycle: resource cannot depend on itself.

7 Unnecessary depends_on

- Adding `depends_on` everywhere reduces parallelism.
- Makes the graph harder to understand.
- Can increase apply time and create unnecessary blast radius.
- Use it only when there is a real dependency not captured by references.

8 Blast Radius of Dependencies

- Destroying or replacing a resource can affect all resources that depend on it.
- Understand the dependency chain before making changes in production.
- Use `terraform plan` to review the full impact.
- Design modules to reduce unnecessary dependencies.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

03



State as Production Infrastructure

1 What is Terraform State?

- Terraform state is a record of the **real infrastructure** that Terraform is managing.
- It maps configuration to real resources and their current state.
- Terraform uses the state to know what to create, update, or destroy.
- State is **critical production data** and must be protected, backed up and monitored.

2 State Structure (Simplified)

```
{
  "version": 4,
  "terraform_version": "1.6.0",
  "serial": 23,
  "lineage": "f1a2b3c4-d123-4567-890a-bcdef1234567",
  "resources": [
    {
      "mode": "managed",
      "type": "aws_instance",
      "name": "web",
      "provider": "provider[\"registry.terraform.io/hashicorp/aws\"]",
      "instances": [ { "attributes": { ... } } ]
    }
  ]
}
```

Key Fields

- version – state format version
- serial – increments on every change
- lineage – unique ID for the state
- resources – tracked resources

3 Resource Addresses

Each resource in state has a unique address.

```
aws_instance.web
module.network.aws_vpc.main
module.app.aws_instance.frontend[0]
module.db.aws_db_instance.this
```

- Addresses identify resources in the state.
- Changing a resource name or moving to a module changes the address (use **moved blocks** or state mv).
- The address is used for tracking, imports, and updates.

4 Lineage and Serials

- Lineage: unique ID for the state. Helps prevent accidental use of an old or wrong state.
- Serial: increments with every successful state change.
- If you copy or restore a state, lineage and serial help Terraform detect mismatches.

```
"lineage": "f1a2b3c4-d123-4567-890a-bcdef1234567"
"serial": 23
```

5 Sensitive Data in State

- State can contain sensitive values (passwords, keys, tokens, private IPs, etc.).
- Always use remote backends with encryption.
- Restrict access using IAM and least privilege.
- Use **sensitive = true** to reduce exposure in outputs, but the value can still exist in the state.



Never store state in public repositories or local machines in production.

6 Drift and State Refresh

- Configuration drift: real infrastructure has changed outside of Terraform.
- **terraform plan** automatically refreshes state (reads real infrastructure) to detect drift.
- Drift can be intentional or accidental.



7 State Corruption

- Can happen due to interrupted writes, manual edits, or backend issues.
- Symptoms: plan errors, missing resources, invalid JSON, resources recreated unexpectedly.



If state is corrupted:

- Stop making changes.
- Restore from a backup.
- Do not manually edit unless you fully understand.
- Verify with **terraform plan** after recovery.

8 Locking, Backup and Recovery

- Use remote backends with state locking (e.g. S3 + DynamoDB, Azure Storage, GCS, Terraform Cloud).
- Locking prevents multiple engineers from making changes at the same time.
- Take regular backups of the state.
- Test state recovery as part of your disaster recovery plan.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



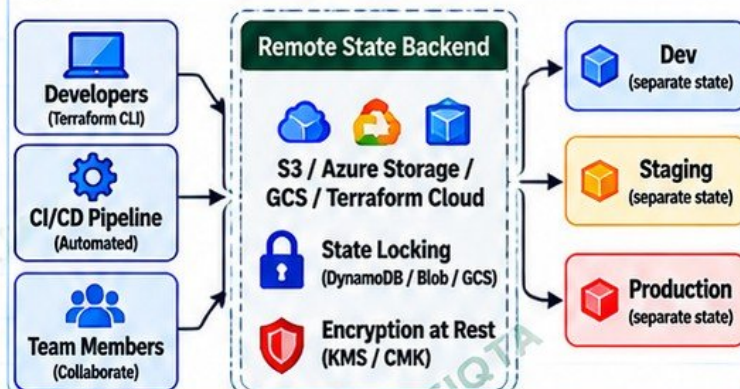
@veriqta

Remote State Architecture at Scale

1 Why Remote State?

- Stores state in a **shared, central** backend.
- Enables team **collaboration** and consistent state.
- Provides **locking** to prevent concurrent changes.
- Supports **encryption**, access control and auditing.
- Required for **production** and multi-environment Terraform.

2 Remote State Architecture



3 Backend Configuration Example (AWS S3)

```

terraform {
  backend "s3" {
    bucket      = "veriqta-terraform-state"
    key         = "prod/network/terraform.tfstate"
    region      = "us-east-1"
    dynamodb_table = "veriqta-terraform-locks"
    encrypt     = true
  }
}
  
```

4 State Isolation and Partitioning

- Use separate state files per environment.
- Separate state per domain (network, compute, data, etc).
- Keep state small and focused.
- Avoid a single global state for the entire organization.

```

s3://veriqta-terraform-state/
├── dev/
├── staging/
├── prod/
│   ├── network/terraform.tfstate
│   ├── compute/terraform.tfstate
│   └── data/terraform.tfstate
  
```

Benefits

- ✓ Smaller blast radius
- ✓ Faster plans
- ✓ Easier access control
- ✓ Independent teams
- ✓ Reduced risk

5 Locking Mechanism

- Prevents multiple engineers from applying changes at the same time.
- DynamoDB (AWS), Blob (Azure) or GCS (Google Cloud) used for locking.
- Locks are automatically acquired and released.
- If a lock is stuck, investigate before force-unlocking.



Do not force-unlock unless you are sure no other operation is running.

6 Security and Access Boundaries

- Use IAM/Service Accounts with least privilege.
- Restrict access to state buckets and lock tables.
- Enable encryption at rest (KMS/CMK).
- Enable audit logs for all access.
- Use separate credentials per environment.
- Do not store sensitive values in state.

7 Failure Domains and High Availability

- Use highly available storage (S3, Azure Storage, GCS).
- Enable versioning for state recovery.
- Consider cross-region replication for critical environments.
- Design for regional outages and service limits.
- Understand the impact if the backend or lock service is down.

8 Backend Migrations

- Use `terraform init -migrate-state` to move to a new backend.
- Test migration in a non-production environment.
- Ensure the new backend has the same security and locking.
- Validate state after migration.

```
terraform init -migrate-state
```

```
# migrate existing state to the new backend
```



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Terraform Plan Reasoning



Read plan output like an operator

1 What is Terraform Plan?

- terraform plan compares your configuration with the current state and real infrastructure.
- Shows what actions will be taken before apply.
- Helps you detect unexpected changes, replacements and destructive actions.
- Always review the plan like an operator, not just a developer.

2 Understanding Plan Output

```
# terraform plan
Terraform will perform the following actions:
# aws_instance.web will be created
+ resource "aws_instance" "web" {
+   id           = (known after apply)
+   ami          = "ami-0abc123456789"
+   instance_type = "t3.micro"
+   tags        = {
+     "Name" = "web"
  }
}
```

- + Create
- ~ Update
- Destroy
- /+ Replace (known after apply)
- Unknown value

3 Unknown Values

- Some values are not known during plan time.
- Shown as (known after apply).
- Common for IDs, ARNs, IPs and attributes computed by the provider.
- Normal and expected for computed attributes.

```
+ id           = (known after apply)
+ arn          = (known after apply)
+ public_ip    = (known after apply)
```

4 Replacement Indicators

- A resource will be replaced when an attribute marked ForceNew changes.
- Plan shows -/+ (destroy and create).
- Replacement can cause downtime.
- Check why it is being replaced before applying.

```
# aws_instance.web must be replaced
-/+ resource "aws_instance" "web" {
  ~ ami          = "ami-123" -> "ami-456" # forces replacement
  ~ instance_type = "t3.micro" -> "t3.small"
}
```

5 Computed Attributes

- Attributes set by the provider based on API responses.
- Usually show as (known after apply).
- Examples: id, arn, public_ip, endpoint, timestamps.
- These are not configuration changes and are expected.

```
+ endpoint      = (known after apply)
+ hosted_zone_id = (known after apply)
+ status        = (known after apply)
```

6 Destructive Changes

- Plan will show resources to be destroyed with -.
- Always review the reason (configuration change, dependency, or moved/removed resource).
- Check the blast radius before applying.

```
# aws_s3_bucket.logs will be destroyed
- resource "aws_s3_bucket" "logs" {
-   id      = "my-logs"
-   arn     = "arn:aws:s3:::my-logs"
}
```

Always verify destructive changes. They can lead to data loss.

7 Dependency Propagation

- A change in one resource can trigger changes in dependent resources.
- Plan shows the full chain of affected resources.
- Example: changing a VPC can replace subnets, route tables and dependent instances.
- Understand the impact across the graph.

8 Review Checklist (Operator Mindset)

- ✓ Are there any unexpected creates, updates or destroys?
- ✓ Are there any replacements and why?
- ✓ Are the unknown values expected?
- ✓ What is the blast radius of the changes?
- ✓ Do dependent resources change?
- ✓ Is this safe to apply in production?





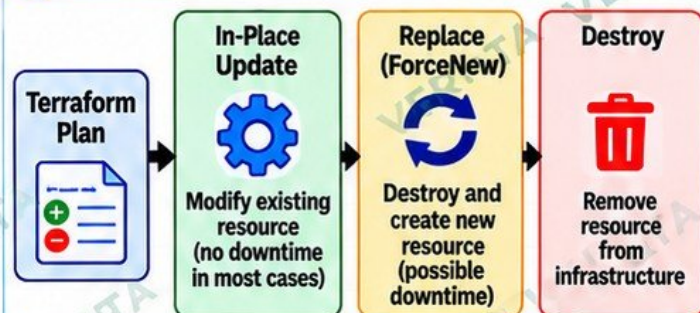
Resource Lifecycle and Replacement

Create, update, destroy and control how resources are replaced

1 Resource Lifecycle in Terraform

- Terraform compares the desired configuration with the current state and decides to:
- Create** – resource does not exist.
- Update** – change can be applied in place.
- Replace** – resource must be destroyed and recreated (**ForceNew**).
- Destroy** – resource removed from configuration.
- Behavior is determined by the provider schema.
- Some attributes can be updated, others require replacement.

2 Lifecycle Flow



3 ForceNew Behavior

- Some attributes cannot be updated in place.
- They are marked as **ForceNew** in the provider.
- Changing these attributes will destroy and recreate the resource.
- Common examples: `name`, `subnet_id`, `instance_type`, `engine_version`.

```
# Terraform plan output
-/+ resource "aws_instance" "web" {
  ~ instance_type = "t3.micro" -> "t3.large" # forces replacement
}
```

4 create_before_destroy

- Creates the new resource before destroying the old one.
- Useful for zero-downtime changes.
- Requires unique names or other configuration adjustments.

```
resource "aws_lb" "example" {
  name = "app-lb"
  lifecycle {
    create_before_destroy = true
  }
}
```

+ Use Cases

- ✓ Load balancers
- ✓ Auto Scaling Groups
- ✓ Databases (with care)
- ✓ Critical production infrastructure

5 prevent_destroy

- Prevents accidental deletion of important resources.
- Plan will fail if Terraform tries to destroy the resource.

```
resource "aws_db_instance" "prod" {
  identifier = "prod-db"
  lifecycle {
    prevent_destroy = true
  }
}
```

! Plan Error
Error: Instance cannot be destroyed due to lifecycle prevent_destroy rule.

6 ignore_changes

- Ignores changes to specific attributes.
- Useful when the value is managed outside Terraform (e.g. auto scaling, external systems).
- Reduces unnecessary updates and drift noise.

```
resource "aws_instance" "web" {
  ami = "ami-12345"
  instance_type = "t3.micro"
  lifecycle {
    ignore_changes = [
      tags,
      user_data
    ]
  }
}
```

! Use with Caution

- Can hide real drift.
- Only ignore attributes that are intentionally managed outside Terraform.
- Still monitor those values.

7 Replacement Risk and Downtime

- Replacements can cause downtime, data loss or service disruption.
- Understand dependencies and blast radius.
- Use **create_before_destroy**, rolling changes, or blue/green patterns to reduce impact.
- Always test in a non-production environment.

8 Best Practices

- ✓ Know which attributes are ForceNew.
- ✓ Use **create_before_destroy** for critical services.
- ✓ Use **prevent_destroy** for important resources.
- ✓ Use **ignore_changes** only when necessary.
- ✓ Review the plan carefully for replacements.
- ✓ Understand the full dependency impact.
- ✓ Test lifecycle changes in staging before production.

07



Drift Detection and Reconciliation

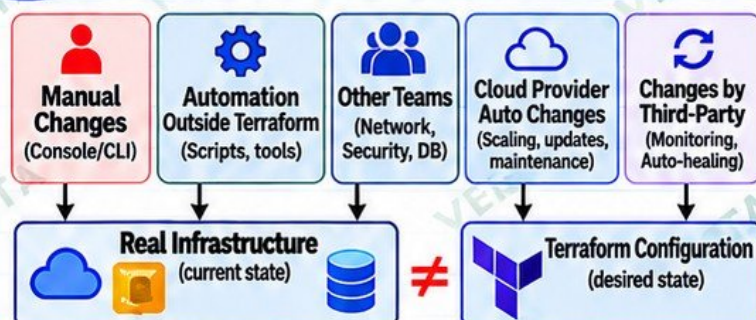


Find, understand and safely reconcile out-of-band changes

1 What is Configuration Drift?

- Configuration drift happens when the **real infrastructure** is different from your Terraform configuration or state.
- Usually caused by out-of-band changes (manual changes, console, CLI, other tools).
- Drift can lead to unexpected behavior, failed applies and security risks.
- You must detect and reconcile drift regularly in production.

2 Sources of Drift (Out-of-Band Changes)



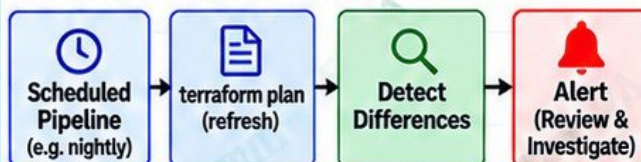
3 Refresh Behavior

- `terraform plan` automatically refreshes the state (reads real infrastructure) before comparing.
- It detects differences between real infrastructure and your configuration.
- It does not fix drift, it only shows what is different.

```
# Refresh and show changes
terraform plan
# Explicit refresh (rarely needed)
terraform apply -refresh-only
```

4 Detecting Drift

- Regularly run `terraform plan` in CI/CD.
- Use scheduled drift detection pipelines.
- Monitor critical resources (IAM, security groups, network, databases, etc.).
- Alert on unexpected changes.



5 Understanding Plan Output for Drift

```
# Example plan showing drift
~ resource "aws_instance" "web" {
  ~ instance_type = "t3.micro" -> "t3.small"
    # changed outside Terraform
  ~ tags = {
    - "Name" = "web"
    + "Env" = "prod" # removed outside Terraform
                  # added outside Terraform
  }
}
```

Plan Symbols

- +** Create (will be added)
- ~** Update (will be changed)
- Destroy (will be removed)
- /+** Replace (destroy and create)
- (known after apply)** Unknown value
- ~** Attribute changed (in-place)

6 Intentional Exceptions

- Some drift is expected and intentional (e.g. auto-scaling, timestamps, external controllers).
- Use `lifecycle ignore_changes` to ignore specific attributes.
- Document intentional exceptions.

```
resource "aws_instance" "web" {
  ami = "ami-12345"
  instance_type = "t3.micro"
  lifecycle {
    ignore_changes = [
      tags["LastUpdated"],
      desired_capacity
    ]
  }
}
```

When to Use

- ✓ Auto-scaling groups
- ✓ Externally managed tags
- ✓ Fields updated by cloud services
- ✓ Non-critical metadata

7 Reconciliation Strategy

- Investigate** – understand why the drift happened.
- Decide** – update Terraform configuration or keep the change.
- Reconcile** – apply changes or import if needed.
- Prevent** – add controls to stop it from happening again.

Reconciliation Options

- Update configuration to match real state
- Import existing changes into state
- Revert the out-of-band change
- Use `ignore_changes` for intentional drift

8 Best Practices

- ✓ Run drift detection regularly.
- ✓ Automate with CI/CD.
- ✓ Alert on critical changes.
- ✓ Use `ignore_changes` only when necessary.
- ✓ Keep documentation of intentional drift.
- ✓ Review and reconcile drift quickly.
- ✓ Implement access controls to reduce manual changes.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Import, Refactoring and State Surgery

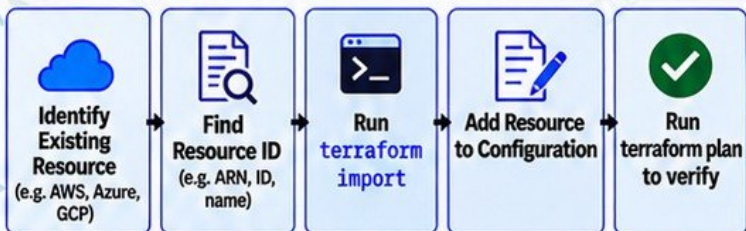


Bring existing infrastructure under Terraform and safely reorganize your state

1 Why Import Existing Infrastructure?

- Allows you to manage existing resources with Terraform.
- Brings manually created resources, console resources or legacy infrastructure under IaC.
- Reduces configuration drift.
- Helps standardize and enforce best practices.
- Useful during migration and brownfield projects.

2 Import Workflow



3 Basic Import Example

```
# Import an existing AWS EC2 instance
terraform import aws_instance.web i-0abc123456789def0
```

Then add the **resource** to your configuration:

```
resource "aws_instance" "web" {
  ami           = "ami-12345678"
  instance_type = "t3.micro"
  tags = {
    Name = "web"
  }
}
```

4 Using Import Blocks (Terraform 1.5+)

- Import blocks allow you to define imports in configuration.
- Useful for repeatable and automated imports.

```
import {
  to = aws_instance.web
  id = "i-0abc123456789def0"
}
```

Benefits

- ✓ Declarative
- ✓ Repeatable
- ✓ Version controlled
- ✓ Works with CI/CD

5 Moved Blocks for Refactoring

- Use moved blocks to rename or move resources without recreating them.
- Preserves the existing infrastructure.

```
moved {
  from = aws_instance.web
  to   = module.compute.aws_instance.web
}
```

6 Resource Renaming and Module Refactoring

- Use moved blocks when renaming resources or moving them into modules.
- Avoids destroy and recreate.
- Works for both root modules and child modules.

```
# Move resource to a new module
moved {
  from = aws_instance.web
  to   = module.web.aws_instance.web
}
```

7 State Surgery with CLI

- Use **terraform state** commands for advanced changes.
- Useful when refactoring or fixing state issues.

```
terraform state list # List resources in state
terraform state show # Show details of a resource
terraform state mv   # Move resource in state
terraform state rm    # Remove resource from state
```



Use state commands carefully. Always take a backup before making changes.

8 Recovery Safeguards

- Always back up state before making changes.
- Use **remote backend** with versioning (S3, Azure, GCS).
- Test refactoring in a non-production environment.
- Verify with **terraform plan** after changes.
- Keep a record of state operations.

Best Practices



- ✓ Backup state before any state surgery.
- ✓ Use moved blocks instead of manual state edits.
- ✓ Test all changes in staging.
- ✓ Verify no unintended changes with **terraform plan**
- ✓ Document what was changed and why.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Production Module Architecture

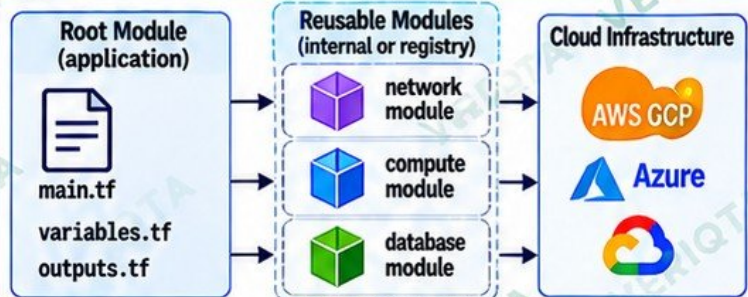


Design scalable, reusable and maintainable Terraform modules

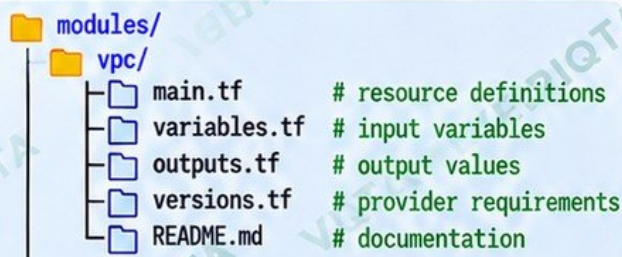
1 What is a Terraform Module?

- A module is a reusable set of Terraform configuration files.
- Helps you package, abstract and reuse infrastructure code.
- Improves consistency, maintainability and collaboration.
- Modules can be local, private (registry) or public (Terraform Registry).

2 Module Architecture Overview



3 Module Structure



4 Module Boundaries and Contracts

- Define clear boundaries and responsibilities.
- Use input variables and output values as the contract.
- Keep modules focused on a single concern.
- Avoid mixing unrelated resources.
- Clearly document expected inputs and outputs.



5 Module Composition

- Combine multiple modules to build complex infrastructure.
- Use a clear hierarchy (root → shared → environment → app).
- Pass outputs from one module as inputs to another.

```
module "vpc" {
  source = "../modules/vpc"
  vpc_cidr = "10.0.0.0/16"
}

module "ec2" {
  source = "../modules/compute"
  subnet_ids = module.vpc.public_subnet_ids
  vpc_id = module.vpc.vpc_id
}
```

6 Versioning and Dependency Direction

- Use semantic versioning for reusable modules.
- Keep dependencies flowing in one direction (root → shared → environment → application).
- Avoid circular dependencies.
- Pin module versions for stability.

```
module "vpc" {
  source = "git::https://github.com/veriqta/terraform-modules//vpc"
  version = "1.2.0" # use a specific version
}
```

7 Reusable vs Application-Specific Modules

✓ Reusable Modules (generic, shareable)

- VPC, networking, IAM
- Security groups
- Logging, monitoring
- Well-documented
- Used across multiple projects

⚙️ Application-Specific Modules (project or environment specific)

- Application servers
- Databases with custom configs
- Environment-specific settings
- Tighter coupling to business needs
- May not be reusable across projects

8 Best Practices

- ✓ Keep modules small and focused.
- ✓ Define clear input and output contracts.
- ✓ Use consistent naming and tagging.
- ✓ Version reusable modules.
- ✓ Avoid deep abstraction (do not hide important details).
- ✓ Test modules before using in production.
- ✓ Document usage with examples.
- ✓ Prefer composition over inheritance.



Multi-Account and Multi-Environment Design



Design secure, scalable and isolated environments with Terraform

1 Why Multi-Account and Multi-Environment?

- Isolates workloads and reduces risk.
- Enforces security and compliance boundaries.
- Prevents accidental changes across environments.
- Allows different configurations, permissions and resources per environment.
- Supports scalable and production-ready operations.

3 Provider Aliases for Multiple Accounts

- Use provider aliases to manage multiple accounts or subscriptions.
- Each alias uses different credentials.
- Keep credentials secure (IAM roles, environment variables or secret manager).

```
provider "aws" {
  alias   = "dev"
  region = "us-east-1"
  assume_role {
    role_arn = "arn:aws:iam::111111111111:role/TerraformRole"
  }
}

provider "aws" {
  alias   = "prod"
  region = "us-east-1"
  assume_role {
    role_arn = "arn:aws:iam::222222222222:role/TerraformRole"
  }
}
```

5 State Separation

- Use separate state files and backends for each environment.
- Keep state in remote backends (e.g. S3, Azure Storage, GCS).
- Enable state locking and versioning.

```
terraform {
  backend "s3" {
    bucket = "veriqta-terraform-state"
    key    = "prod/terraform.tfstate" # use dev/stage/prod
    region = "us-east-1"
    dynamodb_table = "terraform-locks"
    encrypt    = true
  }
}
```

7 Account/Subscription/Project Boundaries



AWS

- Accounts for isolation
- Use Organizations
- Apply SCPs
- Separate IAM roles



Azure

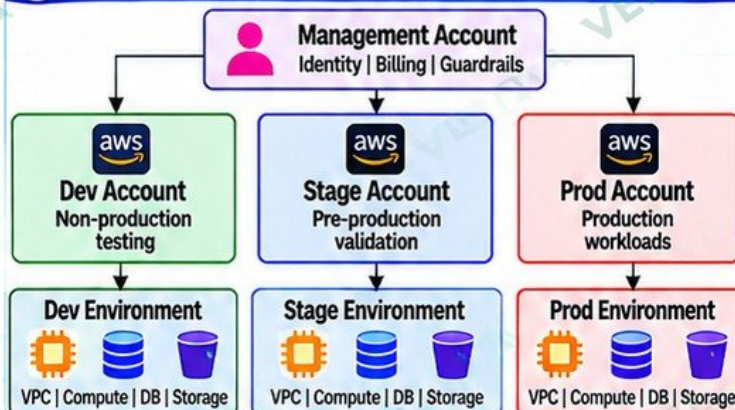
- Subscriptions for isolation
- Use Management Groups
- Apply Policies
- Separate Service Principals



GCP

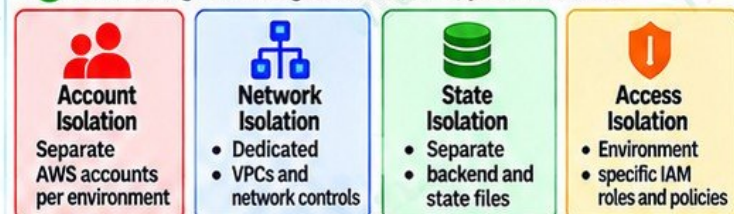
- Projects for isolation
- Use Organization Policies
- Apply IAM policies
- Separate Service Accounts

2 Reference Architecture (AWS Example)



4 Environment Isolation Strategy

- ✓ Separate accounts/subscriptions per environment.
- ✓ Use different VPCs, security groups, IAM roles and policies.
- ✓ Separate state files for each environment.
- ✓ Apply least privilege access.
- ✓ Use tagging and naming conventions.
- ✓ Enforce organization guardrails (SCPs, policies, RBAC).



6 Environment Promotion Models



8 Best Practices

- ✓ Use separate accounts/subscriptions for dev, stage and prod.
- ✓ Keep state files completely isolated.
- ✓ Use provider aliases or workspaces appropriately.
- ✓ Automate environment promotion with CI/CD.
- ✓ Enforce naming, tagging and policy standards.
- ✓ Use least privilege access.
- ✓ Monitor and audit changes across environments.
- ✓ Regularly review access and credentials.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Terraform CI/CD and Safe Delivery



From code to production with security, approvals and control

1 What is Terraform CI/CD?

- Automates the safe and consistent deployment of infrastructure using Terraform.
- Follows a pipeline: validate → lint → security → plan → review → approval → apply → verify.
- Reduces human error and enforces best practices.
- Uses automated checks, approvals and controls to protect production.

2 CI/CD Pipeline Flow



3 Example Pipeline (GitHub Actions)

```

name: Terraform CI/CD
on:
  push:
    branches: [ main ]
jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Setup Terraform
        uses: hashicorp/setup-terraform@v3
      - name: Validate
        run: terraform validate
      - name: Lint (tflint)
        run: tflint --init && tflint
      - name: Security Scan (tfsec)
        run: tfsec
      - name: Plan (immutable plan file)
        run: terraform plan -out=tfplan
      - name: Upload Plan
        uses: actions/upload-artifact@v4
        with:
          name: tfplan
          path: tfplan
  
```

4 Immutable Plans

- Generate a plan file and store it as an artifact.
- The exact plan file is applied after approval.
- Prevents changes between review and apply.

Benefits

- Ensures what was reviewed is what gets applied
- Prevents tampering
- Improves auditability
- Reduces risk in production

```

# Create and save an immutable plan
terraform plan -out=tfplan

# Apply the exact same plan file
terraform apply tfplan
  
```

5 Concurrency Controls

- Prevent multiple pipeline runs from applying at the same time.
- Use environment locks or CI/CD concurrency settings.

```

concurrency:
  group: terraform-prod
  cancel-in-progress: false
  
```

Why It Matters

- Avoids conflicting changes
- Protects remote state
- Prevents accidental overwrites
- Keeps infrastructure stable

6 Protected Production Applies

- Require manual approval for production.
- Use branch protection rules (e.g. only main branch).
- Restrict who can approve (e.g. specific team or role).
- Use environment level protections in CI/CD.



Production Safeguards

- Manual approval required
- Restricted to authorized users
- Runs only from protected branches
- Full audit trail of changes

7 Verification After Apply

- Run checks to confirm resources are healthy.
- Test endpoints, services and key infrastructure components.

Examples

- Check resource status
- Test application endpoints
- Verify DNS, load balancers, databases
- Monitor logs and metrics

8 Best Practices

- Automate the full pipeline.
- Use immutable plan files.
- Implement security scanning.
- Require manual approval for production.
- Use concurrency controls.
- Protect remote state and apply permissions.
- Use environment-specific pipelines (dev, stage, prod).
- Keep pipelines simple and reliable.
- Monitor and alert on pipeline failures.
- Document the process.





IAM, Secrets and Terraform Security



Secure your infrastructure and protect sensitive data with Terraform

1 Why Security Matters in Terraform?

- Protects sensitive data and credentials.
- Prevents unauthorized access and changes.
- Reduces the risk of state and secret leakage.
- Enforces least privilege and compliance.
- Ensures **auditability** and accountability.
- Critical for multi-team and production environments.

2 Key Security Areas

 Least Privilege Minimal permissions	 Workload Identity Secure auth for workloads	 State Exposure Protect state files	 Secret Leakage Prevent accidental leaks	 Provider Credentials Secure provider auth	 CI Identities Secure CI/CD access	 Auditability Track and monitor changes
--	--	---	--	--	--	---

3 Least Privilege IAM (Example: AWS)

```
resource "aws_iam_role" "terraform" {
  name = "terraform-role"
  assume_role_policy = jsonencode({
    Version = "2012-10-17"
    Statement = [{
      Action = "sts:AssumeRole"
      Effect = "Allow"
      Principal = {
        Service = "ec2.amazonaws.com"
      }
    }]
  })
}
```

Best Practices

- ✓ Grant only required permissions.
- ✓ Use IAM roles instead of long-term keys.
- ✓ Use separate roles per environment.
- ✓ Apply conditions and resource restrictions.

4 Managing Secrets Securely

Use environment variables

```
provider "aws" {
  region = var.region
}
```

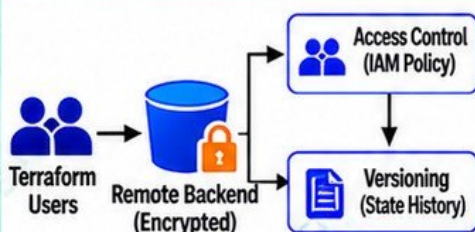
Use a secret from a secrets manager

```
data "aws_secretsmanager_secret_version"
  "db_password" {
    secret_id = "prod/db/password"
  }
```

Security Tips

- ✓ Never hardcode secrets in code.
- ✓ Use environment variables or secret managers (AWS Secrets Manager, Azure Key Vault, GCP SM).
- ✓ Mark sensitive values in outputs.
- ✓ Restrict access to secrets.

5 Protecting Terraform State



Best Practices

- ✓ Use remote backends (S3, Azure Storage, GCS, Terraform Cloud).
- ✓ Enable encryption (at rest and in transit).
- ✓ Use strict IAM policies.
- ✓ Enable state versioning and locking.

6 CI/CD Identities and Authentication

Example: GitHub Actions with OIDC (AWS)

```
provider "aws" {
  region = "us-east-1"
  assume_role {
    role_arn = "arn:aws:iam::123456789012:role/github-oidc-role"
    github_oidc_role = "github-oidc-role"
    web_identity_token_file = "/var/run/secrets/oidc/token"
  }
}
```

Best Practices

- ✓ Use workload identity (OIDC, federated identity).
- ✓ Avoid static credentials.
- ✓ Use least privilege roles for CI/CD.
- ✓ Restrict access to specific repositories and branches.

7 Sensitive Values and Outputs

```
resource "aws_db_instance" "db" {
  identifier = "prod-db"
  password = var.db_password
}

output "db_endpoint" {
  value = aws_db_instance.db.endpoint
  sensitive = true
}
```

Key Points

- ✓ Use `sensitive = true` for sensitive outputs.
- ✓ Avoid printing secrets in logs.
- ✓ Use variables for sensitive values.
- ✓ Be careful with plan/apply output in CI/CD.

8 Auditability and Compliance

 AWS CloudTrail Track API calls and changes	 Azure Monitor Audit resource changes	 GCP Cloud Audit Logs Log all admin activities	 Terraform Logs Track plan and apply actions
--	---	--	--

Best Practices

- ✓ Enable audit logging in your cloud provider.
- ✓ Monitor and alert on unauthorized changes.
- ✓ Keep a record of who made changes and when.
- ✓ Integrate logs with a SIEM for compliance.
- ✓ Regularly review access policies and credentials.

13



Policy as Code and Infrastructure Governance



Enforce standards, security and compliance with Terraform

1 What is Policy as Code?

- Define and enforce rules for your infrastructure using code.
- Prevent insecure, non-compliant or costly configurations.
- Provide automated guardrails and consistent standards.
- Supports security, cost, compliance and operational best practices.

2 Governance Layers



3 Example: Policy with Terraform (OPA - Sentinel/Conftest)

Example: OPA Rego Policy (Require Tags)

```

package terraform.deny

deny[msg] {
  input.resource_changes[_].type ==
    "aws_instance"
  not input.resource_changes[_].change.
    after.tags["Environment"]
  msg = "EC2 instances must have an
    Environment tag"
}
  
```

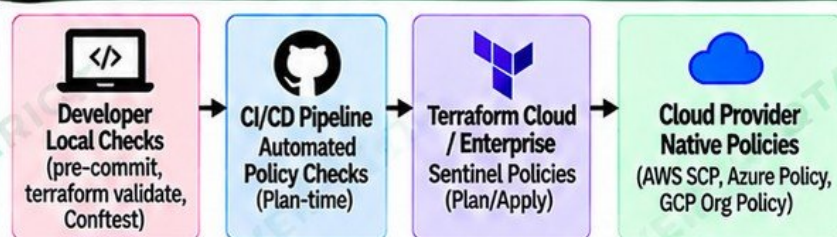
What it does?

- ✓ Denies EC2 instances without an Environment tag.
- ✓ Enforces tagging standards automatically.
- ✓ Can be used with OPA, Conftest or Terraform Cloud policy sets.

4 Common Policy Scenarios

- | | |
|----------------------------------|---|
| Security Baselines | <ul style="list-style-type: none"> • No public S3 buckets • Enforce encryption • Block insecure instance types |
| Tagging Standards | <ul style="list-style-type: none"> • Require mandatory tags (Environment, Owner, CostCenter) • Enforce tag format and values |
| Cost Controls | <ul style="list-style-type: none"> • Restrict expensive instance types • Enforce approved regions • Set resource limits and quotas |
| Prohibited Configurations | <ul style="list-style-type: none"> • Block public IPs • Prevent unencrypted storage • Disallow risky services |

5 Enforcement Points



6 Exception Handling

- | | |
|-----------------|--|
| Document | <ul style="list-style-type: none"> • Business justification • Risk assessment • Approval process |
| Approve | <ul style="list-style-type: none"> • Time-bound exceptions • Limited scope • Management approval |
| Monitor | <ul style="list-style-type: none"> • Regular review and audit • Ensure exceptions are removed • Track in a central system |

7 Example: Terraform Cloud Sentinel Policy

```

import "tfplan/v2" as tfplan

main = rule {
  all tfplan.resource_changes as rc {
    rc.type is "aws_s3_bucket" implies
      rc.change.after.server_side_encryption_configuration is not null
  }
}
  
```

What it does?

- ✓ Ensures all S3 buckets have encryption enabled.
- ✓ Runs automatically during terraform plan in Terraform Cloud.
- ✓ Blocks non-compliant configurations.

8 Best Practices

- ✓ Start with foundational security and tagging policies.
- ✓ Use policy as code early in the development lifecycle.
- ✓ Combine multiple policy tools (OPA, Sentinel, native).
- ✓ Keep policies versioned and reviewed.
- ✓ Implement cost controls and resource limits.
- ✓ Provide clear exception processes.
- ✓ Regularly audit and monitor compliance.
- ✓ Educate teams and document standards.



Provider Internals, Versions and Failure Modes

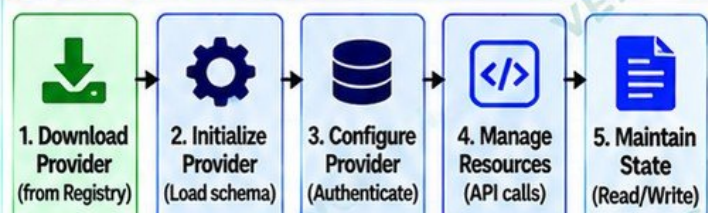


Understand how providers work, manage versions and handle real-world failures

1 What is a Terraform Provider?

- A provider is a plugin that allows Terraform to interact with APIs of cloud platforms and services.
- Translates your configuration into API calls.
- Handles resource schema, validation and state management.
- Each provider has its own release cycle and behavior.
- Understanding provider internals helps you troubleshoot issues and use Terraform effectively.

2 Provider Lifecycle



3 Provider Configuration and Version Pinning

```

terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0" # allow 5.x
    }
  }
}

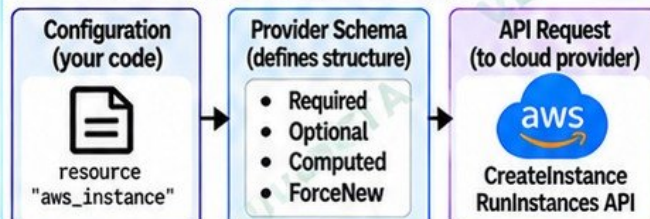
provider "aws" {
  region = "us-east-1"
}
  
```

Version Constraints

- ✓ Use version constraints to avoid breaking changes.
- ✓ Use ~> for minor version updates.
- ✓ Keep providers up to date but test before upgrading.
- ✓ Provider versions are stored in `.terraform.lock.hcl` (lock file).

4 Provider Schema Behavior

- Defines available resources, data sources and arguments.
- Schema tells Terraform how to validate and plan changes.
- Attributes can be Required, Optional or Computed.
- Computed attributes are known only after apply.



5 Lock File (.terraform.lock.hcl)

```

provider "registry.terraform.io/hashicorp/aws" {
  version      = "5.24.0"
  constraints = "~> 5.0"
  hashes = [
    "h1:1234567890abcdef...",
    "zh:abcdef123456..."
  ]
}
  
```

Why Use Lock File?

- ✓ Pins exact provider versions and checksums.
- ✓ Ensures consistent versions across team and environments.
- ✓ Prevents unexpected changes.
- ✓ Committed to version control.

6 Common Failure Modes

- Authentication Failures**
 - Invalid credentials
 - Expired tokens
- API Throttling**
 - Rate limits exceeded
 - Too many concurrent requests
- Eventual Consistency**
 - Resource not immediately available after creation
- Provider Bugs**
 - Unexpected behavior or errors
- Schema Changes**
 - Breaking changes in new versions
- Network and API Errors**
 - Transient network issues
 - Cloud provider API outages

7 Provider Upgrades

- 1 Check release notes for breaking changes.
- 2 Test in a non-production environment.
- 3 Update version constraint in `required_providers`.
- 4 Run `terraform init` to download the new version.
- 5 Review the plan for unexpected changes.
- 6 Apply in a controlled manner (use CI/CD and approvals).

Upgrade Strategy

- ✓ Always test before production.
- ✓ Upgrade in stages.
- ✓ Keep rollback plan (downgrade version).
- ✓ Monitor for provider issues after upgrade.

8 Best Practices

- ✓ Pin provider versions and commit the lock file.
- ✓ Use least privilege credentials for providers.
- ✓ Understand provider schema before using new resources.
- ✓ Handle API throttling with retries and backoff.
- ✓ Design for eventual consistency (use `depends_on`, `timeouts`).
- ✓ Test provider upgrades in a non-production environment.
- ✓ Monitor provider and API changes (release notes).
- ✓ Use CI/CD to validate plans and catch provider issues early.
- ✓ Keep providers up to date for security and new features.



Terraform at Enterprise Scale



Design, operate and scale Terraform for large, complex organizations

1 What Does Enterprise Scale Mean?

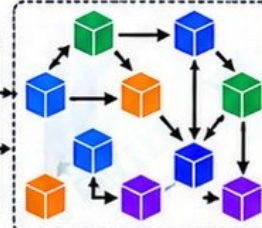
- Hundreds to thousands of resources.
- Multiple teams and environments.
- Large and complex dependency graphs.
- Strict security, compliance and governance.
- Long-term maintainability and clear ownership.
- Integration with enterprise processes and tools.

2 Large Terraform Graphs

Configuration
(Modules + Resources)



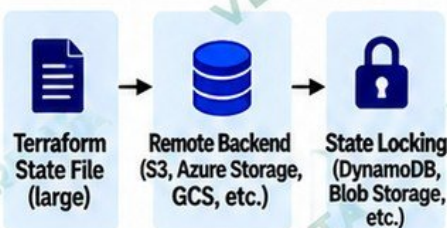
Dependency Graph



Challenges

- ❗ Complex dependencies
- ❗ Slower plans and applies
- ❗ Harder to troubleshoot
- ❗ Changes can impact many resources

3 State Size and Management



Best Practices

- ✓ Use remote backends.
- ✓ Enable state locking.
- ✓ Keep state files small and modular.
- ✓ Regularly clean up unused resources.
- ✓ Use workspaces or separate state per environment.

4 API Limits and Throttling



Mitigation Strategies

- ✓ Use controlled parallelism.
- ✓ Implement retries and exponential backoff.
- ✓ Understand provider-specific API limits.
- ✓ Stagger large changes and use targeted applies.

5 Parallelism and Plan Duration

```
# Set parallelism (default is 10)
terraform apply \
  -parallelism=20
```

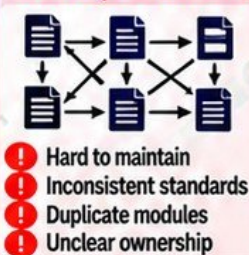
```
# Target specific modules
terraform plan \
  -target=module.network
```

Key Considerations

- ✓ Higher parallelism can reduce plan/apply time.
- ✓ Too high can hit API limits and cause failures.
- ✓ Tune parallelism based on environment and provider.
- ✓ Use `-target` carefully for large environments.

6 Module Sprawl and Repository Strategy

Module Sprawl (Problem)

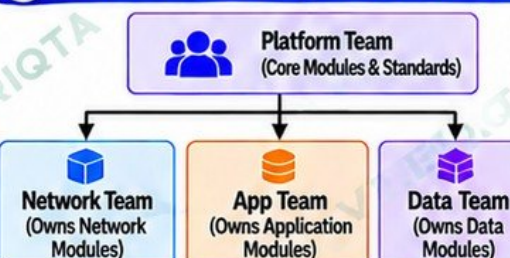


- ❗ Hard to maintain
- ❗ Inconsistent standards
- ❗ Duplicate modules
- ❗ Unclear ownership

Structured Repository Strategy



7 Ownership and Organizational Boundaries



Best Practices

- ✓ Define clear ownership per team and module.
- ✓ Set boundaries and responsibilities.
- ✓ Use consistent standards and templates.
- ✓ Implement code review and governance.
- ✓ Avoid cross-team changes without approval.

8 Overall Best Practices

- ✓ Use modular architecture with clear boundaries.
- ✓ Keep state files small and isolated per environment.
- ✓ Control parallelism and handle API limits.
- ✓ Implement retry and backoff strategies.
- ✓ Use a structured repository strategy.
- ✓ Define ownership and governance processes.
- ✓ Monitor plan and apply duration.
- ✓ Standardize modules, naming, tagging and policies.
- ✓ Integrate with CI/CD and policy as code.
- ✓ Regularly review and refactor as you grow.

16



Failure Recovery and Interrupted Applies

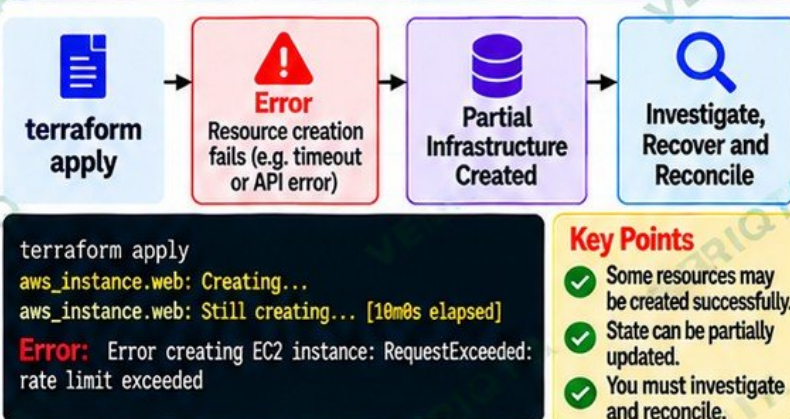


Handle failures, recover safely and keep your infrastructure consistent

1 What Causes Interrupted Applies?

- Partial applies**
Some resources created, others failed.
- Timeouts**
Long running operations exceed limit.
- Provider/API failures**
Network issues, rate limits or cloud service errors.
- Stale state locks**
Apply fails and lock remains.
- Inconsistent state**
State file does not match real infrastructure.

2 Example: Partial Apply Failure



3 Stale Locks and Lock Recovery



4 Common Failure Scenarios

- Timeouts**
 - Increase timeouts for long operations.
 - Check provider and cloud service limits.
- API Failures**
 - Rate limits, throttling or transient errors.
 - Use retries and backoff.
- Partial Applies**
 - Some resources created, others failed.
 - Re-run apply after fixing the issue.
- Stale Locks**
 - Lock remains after a failed or canceled run.
 - Check and force-unlock only when safe.
- Inconsistent State**
 - State does not match real infrastructure.
 - Use terraform refresh and import if needed.

5 Recovery Steps (Step by Step)

- 1 Identify the failure cause (logs, error messages).
- 2 Check what resources were created.
- 3 Verify state status and lock.
- 4 Fix the underlying issue (e.g. quota, timeout, config).
- 5 If locked, confirm no active process and force-unlock if safe.
- 6 Run terraform refresh to sync state with real infrastructure.
- 7 Plan again to review changes.
- 8 Apply and verify that infrastructure is consistent.

6 Useful Commands

```
# Check current state
terraform show
# Refresh state
terraform refresh
# Check lock (backend specific)
terraform force-unlock <LOCK_ID>
# Re-run plan
terraform plan
# Apply again
terraform apply
```

7 Best Practices

- ✓ Use remote state locking (S3, Azure, GCS, Terraform Cloud).
- ✓ Understand provider and API limits.
- ✓ Set appropriate timeouts.
- ✓ Use retries and exponential backoff.
- ✓ Always investigate before force-unlock.
- ✓ Keep state and real infrastructure consistent.
- ✓ Monitor apply progress in CI/CD.
- ✓ Document failure scenarios and recovery steps.
- ✓ Regularly test recovery procedures.

8 Safeguards and Prevention



17



Zero-Downtime Infrastructure Changes



Make safe changes without service interruption using Terraform

1 What is Zero-Downtime Infrastructure?

- ✓ Make infrastructure changes without interrupting user access or services.
- ✓ Replace or update resources safely.
- ✓ Use rolling transitions and immutable infrastructure where possible.
- ✓ Plan changes to avoid dependency failures.
- ✓ Keep applications highly available during infrastructure updates.

2 Replacement Sequencing

1. Create New



Provision new infrastructure alongside existing.

2. Shift Traffic



Move traffic gradually using load balancer or DNS.

3. Validate



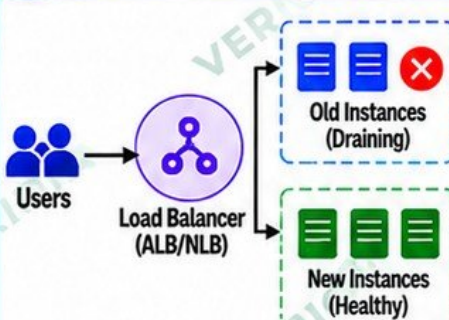
Check health, metrics and application functionality.

4. Remove Old



Decommission the old infrastructure after validation.

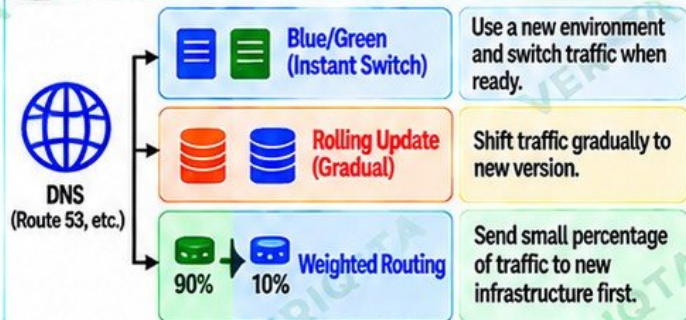
3 Load Balancer Rolling Transition (Example)



Key Points

- ✓ Create new instances before removing old ones.
- ✓ Use health checks.
- ✓ Enable connection draining.
- ✓ Shift traffic gradually (e.g. weighted target groups).
- ✓ Remove old instances after validation.

4 DNS and Traffic Management



5 Database Changes



Best Practices

- ✓ Use read replicas for testing and migrations.
- ✓ Apply schema changes with backward compatibility.
- ✓ Use migration tools (e.g. expand and contract).
- ✓ Avoid long-running locks.
- ✓ Validate before switching traffic.

6 Example: Safe Replacement with Terraform

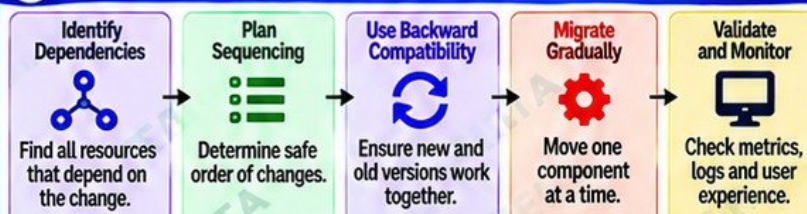
```
resource "aws_launch_template" "app" {
  name_prefix = "app-v2-"
  image_id    = "ami-12345678"
  instance_type = "t3.large"
}

resource "aws_autoscaling_group" "app" {
  name = "app-asg"
  launch_template {
    id      = aws_launch_template.app.id
    version = "$Latest"
  }
  min_size = 2
  max_size = 4
  health_check_type = "ELB"
}
```

Key Points

- ✓ Create new launch template (versioned).
- ✓ Update Auto Scaling Group to use new version.
- ✓ Use health checks and connection draining.
- ✓ Verify instances are healthy before removing old ones.
- ✓ Terraform handles the rolling replacement safely.

7 Dependency-Safe Migration Strategy



8 Best Practices for Zero-Downtime Changes

- ✓ Use immutable infrastructure wherever possible.
- ✓ Create new resources before destroying old ones.
- ✓ Use load balancers and connection draining.
- ✓ Implement rolling updates or blue/green deployments.
- ✓ Use DNS strategies for safe traffic switching.
- ✓ Plan database changes carefully with backward compatibility.
- ✓ Monitor health, errors and performance during the change.
- ✓ Automate validation and rollback plans.
- ✓ Always test in a non-production environment first.



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta



Observability and Terraform Operations



Monitor, track and operate Terraform for reliable infrastructure delivery

1 What is Terraform Observability?

- ✓ Visibility into how Terraform is used, what changes are made and how it performs.
- ✓ Track key metrics like apply duration, failure rates, drift and policy violations.
- ✓ Helps you detect problems early, improve reliability and ensure compliance.
- ✓ Supports operational dashboards, alerting and audit trails.

2 Key Metrics to Monitor

 **Apply Duration**
Time taken for plan and apply to complete.

 **Failure Rates**
Percentage of failed plans and applies.

 **Drift**
Difference between state and real infrastructure.

 **Policy Violations**
Resources that breach policy rules.

 **State-Lock Contention**
Frequency and duration of state locks.

 **Change Frequency**
How often changes are made to infrastructure.

 **Audit Trails**
Who made changes, when and what changed.

 **Operational Dashboards**
Centralized view of metrics, logs and alerts.

3 Terraform Operations Workflow



4 Example Operational Dashboard



5 Logs and Audit Trails

Example: Terraform logs (CI/CD)

```

2026-09-26T10:15:23Z [INFO] Starting apply....
2026-09-26T10:15:24Z [INFO] module.vpc: Creating...
2026-09-26T10:16:10Z [ERROR] Error: Provider API rate limit exceeded
2026-09-26T10:16:10Z [INFO] Apply failed
  
```

Example: Audit Log Record

```

{
  "user": "ci-bot",
  "action": "apply",
  "workspace": "prod",
  "timestamp": "2026-09-26T10:15:23Z",
  "status": "failed",
  "changes": "3 to add, 1 to change, 0 to destroy"
}
  
```

Key Audit Information

- ✓ Who made the change (user, service account).
- ✓ When the change happened.
- ✓ Which workspace/environment.
- ✓ What resources changed.
- ✓ Plan and apply results.
- ✓ Link to CI/CD run and approval record.

6 Alerting and Monitoring

-  **Failed Applies** Alert on apply failures and repeated errors.
-  **Long Apply Duration** Alert when apply time exceeds threshold.
-  **State Lock Contention** Alert on long-running locks or frequent lock failures.
-  **Policy Violations** Alert when policies are violated in plan or apply.
-  **Drift Detection** Alert on significant drift between state and real resources.

7 Operational Best Practices

- ✓ Centralize Terraform logs and metrics.
- ✓ Track apply duration, success and failure rates.
- ✓ Monitor state lock contention.
- ✓ Detect and alert on drift regularly.
- ✓ Enforce and monitor policy violations.
- ✓ Maintain complete audit trails.
- ✓ Use dashboards for real-time visibility.
- ✓ Set up alerts and escalation for failures.
- ✓ Review change frequency and trends.
- ✓ Regularly audit access and permissions.

8 Tools and Integrations

-  **Terraform Cloud**
(Built-in run history, metrics and audit logs)
-  **Prometheus + Grafana**
(Custom metrics and dashboards)
-  **ELK / OpenSearch**
(Logs and audit trails)
-  **Cloud Provider Monitoring**
(AWS CloudWatch, Azure Monitor, GCP Operations)
-  **SIEM / Security Tools**
(Compliance and audit integration)

19



Production Incident: Terraform Wants to Destroy Infrastructure



Step-by-step investigation, mitigation, recovery and prevention

1 Incident Trigger: Suspicious Plan

You run terraform plan and see resources will be destroyed that should not be destroyed.



```
# terraform plan
```

```
...
-/+ aws_instance.web (forces replacement)
-/ aws_security_group.app (will be destroyed)
-/ aws_db_instance.main (will be destroyed)
```

Plan: 3 to add, 0 to change, 3 to destroy.

2 Immediate Actions

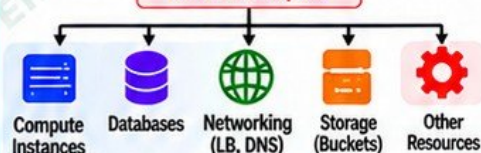
- ✓ Do NOT run terraform apply.
- ✓ Stop any running pipeline.
- ✓ Notify the team and open an incident.
- ✓ Preserve the current state plan output and logs.
- ✓ Check if this is from CI/CD or a local run.
- ✓ Assess potential blast radius before making changes.



3 Determine Blast Radius

- Which resources will be destroyed?
- What environments are affected (dev, staging, production)?
- Will this impact user-facing services?
- Are there dependencies (databases, load balancers, networking)?
- What is the business impact and urgency?

Potential Impact



4 Investigate the Root Cause

	Configuration Changes	Review recent code changes, variables and module updates.
	State Issues	Check for missing or corrupted state, wrong backend or workspace.
	Provider/Version Changes	Look for provider upgrades or behavior changes.
	Variable or Environment Changes	Check .tfvars, environment variables and CI/CD inputs.
	Resource Drift	Compare real infrastructure with state (terraform refresh, plan).
	Module/Code Refactoring	Check for renamed, moved or removed resources.

5 Collect Evidence

- Save the full terraform plan output (-out=planfile).
- Check git history for recent changes.
- Review state file (terraform state list, terraform state show).
- Check provider version and .terraform.lock.hcl.
- Verify variables and workspace.
- Review CI/CD logs and pipeline configuration.
- Compare with cloud provider console (actual resources).

6 Mitigation Options

- Fix the configuration error (e.g. revert recent changes).
- Use -target to limit changes (temporary and careful use only).
- Restore the correct state file (from backup if needed).
- Revert to a known good commit in version control.
- Pin or roll back provider version if a provider issue.
- Validate variables and environment settings.

7 Recovery Steps

- 1 Correct the root cause (code, variables, state, provider).
- 2 Run terraform plan again and verify no unexpected destroys.
- 3 If state was lost or corrupted, restore from backup.
- 4 Apply changes in a controlled manner (small, targeted).
- 5 Monitor the infrastructure after apply.
- 6 Confirm all critical resources are healthy.

8 Verification

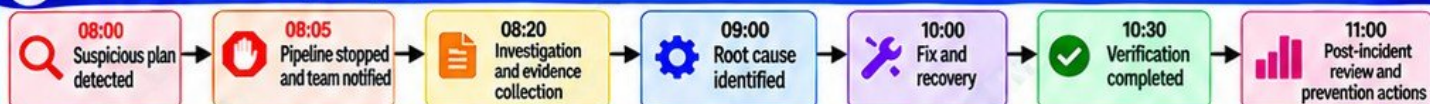
- ✓ Run terraform plan and confirm only expected changes.
- ✓ Verify resources in the cloud console.
- ✓ Check application and service health.
- ✓ Validate no unintended resource destructions occurred.
- ✓ Confirm monitoring, alerts and dependencies are working.



9 Prevention

- ✓ Implement code reviews and approvals for production changes.
- ✓ Use automated policy checks (e.g. OPA, Sentinel) to block unwanted destroys.
- ✓ Enable mandatory plan review in CI/CD.
- ✓ Use separate workspaces and state per environment.
- ✓ Set up alerts for large or destructive plans.
- ✓ Regularly back up your state files.
- ✓ Use version pinning for providers and modules.
- ✓ Maintain clear documentation and runbooks.
- ✓ Perform regular drift detection.

10 Incident Timeline (Example)



@veriqta_academy



@veriqta



@veriqta



@veriqta



@veriqta



@veriqta

20



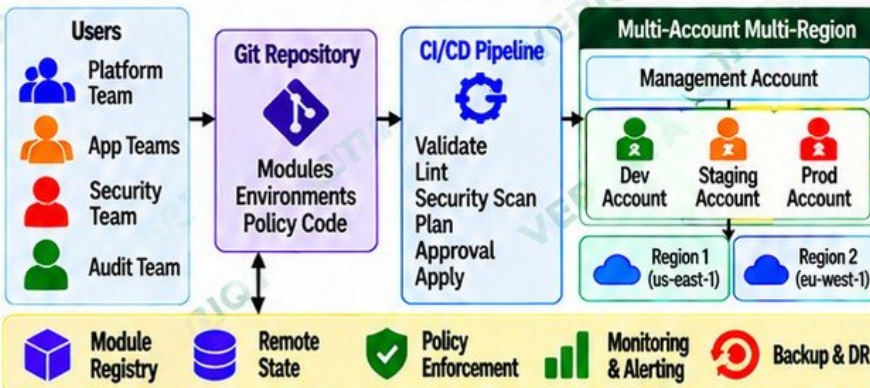
Senior Architecture Case Study: Enterprise Terraform Platform



Design a multi-account, multi-region Terraform operating model
Covering modules, state, CI/CD, IAM, policy, observability, DR, ownership and recovery

1 Architecture Overview

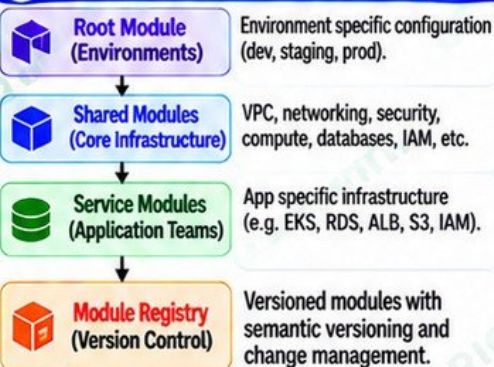
Multi-account, multi-region Terraform platform with centralized governance, secure state, automated CI/CD and full observability.



2 Key Design Principles

- ✓ Scalable multi-account, multi-region design.
- ✓ Reusable, versioned and consistent modules.
- ✓ Secure and centralized remote state.
- ✓ Automated and controlled CI/CD pipelines.
- ✓ Least privilege IAM and workload identity.
- ✓ Policy as code with clear guardrails.
- ✓ Full observability, audit trails and reporting.
- ✓ Disaster recovery and business continuity.
- ✓ Clear ownership and operational boundaries.
- ✓ Safe change management and recovery processes.

3 Terraform Module Strategy



4 State Management

- Remote state in S3 (or equivalent).
- DynamoDB (or equivalent) for locking.
- Separate state per environment and account.
- Encryption at rest and in transit.
- State access with least privilege IAM.
- State backup and versioning.
- Monitoring and alerting on state access and lock contention.

5 CI/CD Pipeline Design

- 1. Code Push**
Developer pushes changes to Git.
- 2. Automated Checks**
Validate, lint, security scan, policy check.
- 3. Terraform Plan**
Generate plan and store as artifact.
- 4. Review and Approval**
Manual or automated approval.
- 5. Terraform Apply**
Apply after approval (controlled by environment and workspace).
- 6. Verify and Notify**
Verify changes and send notifications.

6 IAM and Security

- ✓ Least privilege IAM for users, CI/CD and services.
- ✓ Use roles and workload identity (OIDC).
- ✓ Restrict access to state, modules and resources.
- ✓ Use short-lived credentials.
- ✓ Separate duties (dev, ops, security, audit).
- ✓ Monitor and audit all Terraform operations.

7 Observability and Auditability

- Track apply duration, success and failure rates.
- Monitor drift and policy violations.
- Monitor state lock contention.
- Dashboards (e.g. Grafana, CloudWatch, ELK).
- Centralized audit logs (CI/CD, state access, plan/apply history).
- Alerts for failures, unusual activity and unauthorized changes.

8 Disaster Recovery and Operations

- Regular state backups and versioning.
- Multi-region strategy for critical workloads.
- Documented recovery procedures.
- Clear ownership and on-call responsibilities.
- Runbooks for common and critical failures.
- Regular testing of recovery processes.

9 Example Multi-Account Structure



10 Success Criteria

- Secure, scalable and repeatable platform.
- Minimal manual work and human error.
- Full visibility and auditability.
- Fast and safe delivery of infrastructure.
- Resilient to failures with clear recovery.
- Meets compliance and governance requirements.